

DESIGN DOCUMENT

PS10 – Grid Travel Agent

AIMLCZG557/AECLZG557 | S2 2025–2026 | Group G042

1. Problem Statement

The objective is to design an intelligent routing agent (the knight) that navigates a two-dimensional grid to reach a queen in distress. The grid contains terrain cells — forests and water bodies that yield +5 rewards, fire and wild animals that incur −5 penalties, and mountains that incur −3 penalties. The agent must find the shortest path that simultaneously maximises net reward. The environment is fully observable, so the agent has complete knowledge of the grid before the search begins. The A* Search Algorithm is used to solve this path-finding problem.

2. PEAS Analysis

Component	Description
Performance Measure	Reach the queen • Minimise total cells traversed • Maximise cumulative reward (+5 forest/water, −5 fire/animals, −3 mountain) • Minimise unnecessary node expansions
Environment	Rectangular N×M grid; fully observable, static, discrete, deterministic, single-agent. Cells: Empty, Forest, Water, Fire, Wild Animal, Mountain, Knight (start), Queen (goal).
Actuators	Move Up • Move Down • Move Left • Move Right (4-directional, within grid bounds only)
Sensors	Full grid perception (all terrain locations) • Current position • Goal position • Valid neighbouring cells • Reward values of adjacent terrain

3. Environmental Characteristics

Property	Justification
Fully Observable	The complete grid configuration is known to the agent before search starts.
Deterministic	Each move produces a single, predictable outcome.
Sequential	Each decision affects subsequent states and accumulated rewards.
Static	The grid does not change while the agent is searching.
Discrete	Finite set of grid cells and a finite set of actions.
Single-Agent	Only the knight operates; no adversary or co-operative agent.
Known	Transition rules and reward values are fully specified in advance.

4. Problem Formulation & A* Algorithm

4.1 State Representation, Actions & Cost

A state is a tuple (row, col). The agent moves in four directions provided the destination remains within grid bounds. Each move incurs a step cost of $1 - \text{reward}(\text{cell})$, where $\text{reward}(\text{cell}) \in \{+5, 0, -3, -5\}$. This encodes the dual objective: rewarding cells are cheaper to traverse, penalty cells are costlier.

4.2 A* Evaluation Function

$$f(n) = g(n) + h(n)$$

- $g(n)$ — cumulative step cost from start to node n
- $h(n)$ — Manhattan distance heuristic: $h(n) = |\text{row}_n - \text{row}_{\text{goal}}| + |\text{col}_n - \text{col}_{\text{goal}}|$

- The node with the lowest $f(n)$ is expanded first.

4.3 Heuristic Function Justification

Manhattan distance is used because movement is restricted to four cardinal directions. It is admissible (never overestimates: each step reduces Manhattan distance by at most 1) and consistent / monotone ($h(n) \leq \text{step_cost}(n, n') + h(n')$ for every successor n'). These properties guarantee that A* finds the optimal path — shortest distance with maximum net reward — without re-expanding nodes.

4.4 Data Structures

Structure	Role in A*	Operations & Error Handling
Priority Queue (Min-Heap)	OPEN list — stores nodes pending expansion ordered by $f(n)$	<code>insert(priority, item)</code> → OverflowError if full <code>extract_min()</code> → IndexError if empty <code>peek()</code> → IndexError if empty
Hash Set (Closed Set)	CLOSED list — tracks fully expanded nodes to avoid re-processing	<code>add(state)</code> → TypeError if non-tuple <code>contains(state)</code> , <code>size()</code> , <code>is_empty()</code>
Grid Class (2-D List)	Stores terrain and provides bounds-checked cell access, reward lookup, and neighbour generation	<code>get_cell()</code> / <code>set_cell()</code> → IndexError if OOB <code>get_reward()</code> , <code>get_neighbors()</code> , <code>find_cell()</code>
Dictionary (g_cost map)	Tracks best known g-cost per state for duplicate detection	O(1) average lookup and update

5. Complexity Measurement (Empirical)

Complexity is measured at runtime — not derived theoretically — as required by the assignment:

- Time complexity: measured using Python's `time.perf_counter()` around the A* call. Reported in milliseconds.
- Space complexity: measured using Python's `tracemalloc` module; peak heap allocation is reported in KB.
- Search effort: number of nodes expanded (popped from the OPEN list) is counted and printed.

Sample output on the default 7×7 grid (start (0,0), goal (6,5)):

Metric	Measured Value	Note
Nodes expanded	14	Actual search effort
Execution time	~0.34 ms	<code>perf_counter()</code>
Peak memory	~4.7 KB	<code>tracemalloc</code>

6. Alternate Modelling: Weighted Graph Representation

6.1 Description

Instead of a 2-D grid array, the environment can be modelled as a weighted directed graph $G = (V, E, w)$. Each grid cell (r, c) becomes a vertex $v \in V$. For each pair of adjacent cells, a directed edge $e \in E$ is added with weight $w(e) = 1 - \text{reward}(\text{destination cell})$. The A* algorithm then operates on this graph structure, using an adjacency list for O(1) neighbour lookup.

6.2 Performance Implications

Aspect	Grid Representation (chosen)	Graph Representation (alternate)
Memory	$O(N \times M)$ — stores only cell types in a 2-D list	$O(V + E) = O(N \times M + 4 \times N \times M) \approx 5 \times$ higher overhead due to adjacency structures
Neighbour Generation	O(1) — compute $(r \pm 1, c \pm 1)$ on the fly with bounds check	O(1) — direct adjacency list lookup; no bounds computation needed

Preprocessing	None — grid is ready to search immediately after loading	$O(N \times M)$ build time to construct vertex/edge sets and weight map
Extensibility	Uniform grid required; diagonal moves need code change	Easily extended: add diagonal, teleport, or variable-cost edges without changing the search logic
Scalability	Efficient for dense, regular grids (most game/robot maps)	Better for sparse, irregular maps (road networks, dungeon layouts)
Implementation Complexity	Simple — index arithmetic only; intuitive for rectangular worlds	Higher — requires graph builder, adjacency list class, and weight dictionary in addition to the search code

6.3 Conclusion

The grid representation is preferred for this assignment because the environment is a regular rectangular grid, neighbours are computed in $O(1)$ without preprocessing, and memory overhead is minimal. The graph model would be advantageous for large irregular terrains, non-uniform connectivity (e.g., one-way passages), or when integrating with external graph libraries such as NetworkX for advanced analytics.